

Why do I keep losing?

This is a question that every Dota player has asked themselves. Some players may blame their team, others may accuse the draft, but all players definitely have this question cross their mind at some point.

Dota 2 is an extremely complex game that has many variables that can determine whether you get the +25 or the -25, either way you are still gonna queue next.

Myth or truth: "We already lost in the draft"

There are 127 total unique heroes and 10 are chosen per game, resulting in millions of combinations every single match. This article explores the importance of the draft by training a model with just knowledge about the draft to predict which team wins. Of course this is not a perfect experiment by any means, but I tried my best to make it as fair as possible.

You've probably seen plenty of Dota win prediction models before, but I wanted to do it with a twist. Instead of just passing hero names as labels, I passed their icons instead. My thinking was that maybe those extra channels of data hiding in images, stuff that isn't present in plain text, could hold some pattern that wouldn't be intuitive or obvious to us mere humans. Spoiler alert, that idea took me on a much longer ride than I expected, full of dead ends, bugs, and a few genuinely useful lessons.

Building the dataset

The dataset only includes pro matches, since pro players are expected to have close to perfect play and the best understanding of matchups. That way we can see the true strength of heroes more clearly, since the variance from skill level gets mostly filtered out. I pulled everything from the OpenDota API using their free tier, and grabbed hero icons from the Liquipedia wiki.

Every match becomes an image built out of 10 hero icons arranged in a 2x5 grid, radiant on top, dire on the bottom. I ordered heroes by role (1/5, 2, 4/3) so there's some implicit positional encoding baked into where each icon sits, and I also built in an option to randomize that order for comparison later as an ablation.

Total dataset ended up being 5,939 pro matches, all sitting on the exact same patch, namely the latest one. I kept it locked to one patch on purpose, since hero winrates and matchups shift a lot between patches thanks to balance changes, item reworks, all that stuff. I wanted to keep things as fair as possible.

Hurdle 1: the model that only ever guessed

First attempt was simple. Grab a pretrained ViT, swap the classification head, fine tune it on the hero grid images. Simple in theory.

Reality: test accuracy sat at exactly 0.5285 or 0.4715, epoch after epoch, run after run, no matter which hero ordering I used. Those two numbers immediately looked suspicious, since they summed to exactly 1.0. That's the signature of a model that's just picking one class every time and calling it a day. Worse, train accuracy never rose either. If it had been overfitting, I'd expect train accuracy climbing while test stayed flat. Instead both were frozen near chance, meaning the model wasn't even managing to fit the training set, let alone generalize.

The fix. Turned out the problem was mostly in how the images were built. I was taking a big 2x5 grid of full sized 224x224 hero icons and squashing the whole thing down into a single 224x224 image at the end. Every hero portrait got crushed into a blurry smear only a few pixels wide, basically unrecognizable even to a human eye. I fixed this by resizing each icon to a much smaller, patch aligned cell size before assembling the grid, so nothing gets squished into mush during a final resize. I also made sure normalization actually matched whatever pretrained backbone I was using, since some expect ImageNet mean and std while others expect a flat 0.5/0.5/0.5, and mismatching that alone can quietly wreck a fine tune.

Hurdle 2: making sure the pipeline wasn't lying to me

Even with cleaner images, I didn't fully trust the results yet, so I ran a full audit before touching the model again.

Checked that a model could memorize a tiny 8 example batch. If it can't do that, something upstream is broken and no amount of real training helps.

Verified labels lined up correctly between the raw dataframe and what the dataset's getitem was actually returning.

Checked for duplicate matches, hero id overlap between the two teams (which would signal a parsing bug), and leakage between train, val, and test splits.

Confirmed the "single patch" claim actually held in the data, so patch mixing wasn't secretly polluting anything.

Everything came back clean. That was reassuring and also mildly annoying, since it meant the weirdness I was seeing was a real modeling problem, not a bug I could just squash and move on from.

Hurdle 3: a properly boring baseline

At this point I needed a reference point that had nothing to do with images. So I one hot encoded which heroes were on which team and threw it at a Gradient Boosted Trees model.

Result: around 54.3% accuracy, 0.56 AUC.

Not a huge number, but a real one, and it changed the whole picture. It told me the draft does carry some genuine signal, a modest edge, which lines up with published research on picks only prediction (most work in this space caps out around 55 to 60%). More importantly, this dead simple linear model found signal that every single vision architecture I'd tried so far completely missed. That pointed the finger squarely at the image representation itself, not the underlying task.

Hurdle 4: chasing the vision approach anyway

I wasn't ready to give up on images yet, so I ran a much bigger sweep. ResNet18 pretrained and from scratch, ViT Base, ViT Small, ViT Tiny, DeiT3, Swin Tiny, frozen backbones, fully fine tuned backbones, lower learning rates, more epochs, bigger icon cells to cut down on wasted padding, the works.

This is where the failure mode changed shape. Looking at `resnet18_roles_pretrained_finetune` and `resnet18_roles_scratch` specifically, train accuracy climbed steadily up past 70, even 90%+, while test accuracy stayed pinned around 50 to 55%, and test loss actually started rising over time. That's classic overfitting, the model successfully memorizing exactly which hero combos showed up in which training matches, without learning anything that generalized.

The fix. Added early stopping based on validation loss and some regularization, dropout and weight decay. That cleaned up the memorization nicely, train accuracy stopped running away from test accuracy, but the actual test ceiling never moved. Every architecture, every regularization setting, landed back in that same 50 to 55% band. Regularizing a model that never found the signal in the first place just makes it fail more slowly and honestly, it doesn't make the signal appear.

Hurdle 5: a sneaky split imbalance

While digging through results I noticed something else. Radiant wins more often overall in this dataset (about 52.4%), and my original random train/val/test split let that imbalance drift unevenly across splits purely by chance. My test set at one point had a 56.5% radiant rate, noticeably higher than train's 51.4%. That's a real problem, since a model could look artificially better or worse just by lining up with whatever the test split's majority class happened to be that day, especially since several of my results were sitting right around 53 to 56% anyway.

The fix. Swapped the plain random split for scikit learn's `stratified train_test_split`, which forces every split to preserve the same overall win distribution. Reran the sweep afterward, and the story held up basically unchanged, image models still stuck near chance, GBM and the embedding MLP still landing in the mid 50s. Good news in a roundabout way, it meant the earlier conclusions weren't just an artifact of a lucky or unlucky split.

Hurdle 6: giving the embedding approach its own honest shot

Given how well the one hot GBM performed, I built its neural network cousin, a small MLP with a learned embedding table for each hero id instead of a fixed one hot row, radiant and dire pooled separately then combined before a couple of dense layers.

This landed around 54.5 to 55.5% test accuracy at its best epoch, right in line with the GBM, though it does show the same overfitting shape if you let it keep training past that point (train accuracy climbing into the 70s by epoch 20 while test bounces around in the low 50s). Early stopping on validation accuracy or loss handles that the same way it did for the vision models.

Can we actually trust the GBM and MLP as proper baselines?

Short answer, yes, with a couple of caveats worth being upfront about.

Why they hold up. Both models are working directly on hero ids, the most direct possible representation of "who is on this team," with no lossy intermediate step like image encoding in the way. Two very different model families, one linear and tree based, one a small neural net with learned embeddings, independently converge on almost the same number, 54.3 to 54.5% for GBM and roughly 54.5 to 55.5% for the MLP at its best point. That kind of agreement between structurally different approaches is a good sign the number reflects something real about the data rather than a quirk of one particular model or training run. It also matches published numbers from other picks only prediction work reasonably well, which gives an external anchor beyond just internal consistency.

Where to stay a little careful. The MLP overfits fast if you don't stop it early, so any number quoted from it should come from the best validation epoch, not wherever training happened to end. A single stratified train/test split, even a good one, is still just one draw, and with roughly 890 test matches there's real sampling noise on the order of a percentage point or two. If I wanted to fully lock this down I'd run repeated stratified k fold cross validation on both the GBM and the MLP and report a mean and spread rather than a single number, that would tell me whether 54.5% is a stable estimate or if it could reasonably swing between say 52 and 57% depending on the split. Right now I'm reasonably confident in the direction of the result, draft signal is real but modest, and clearly above the vision models, but I'd hold the exact decimal loosely.

Where the vision models actually looked

Once I had baselines I trusted, I went back to the image models purely to see what they'd been looking at, since accuracy wasn't going to improve from here.

Saliency maps on the fine tuned ResNet and ViT models showed something that lined up perfectly with the accuracy story. Gradient attention was smeared diffusely across the entire 224x224 canvas, including the large gray empty padding area around the actual hero icons, rather than being sharply focused on the portraits themselves. A model that had genuinely learned to key in on hero identity should light up tightly on the icons and mostly ignore blank space. Mine didn't. Which explains the numbers, it never found a strong, localized, hero specific feature to grab onto in the first place.

Where this leaves the scoreboard

Vision models, ViT Base, Small, Tiny, DeiT3, Swin, ResNet18, across pretrained, from scratch, frozen, fine tuned, and regularized variants, all landing around 49 to 53%, indistinguishable from guessing once you account for split imbalance.

Gradient Boosted Trees on one hot hero vectors, around 54.3% accuracy, 0.56 AUC.

Embedding based MLP, around 54.5 to 55.5% at its best epoch, matching the GBM closely.

Put plainly, the draft does carry a small real signal, "we already lost in the draft" has a kernel of truth to it, but it's a modest edge, not a dominant one, and turning that signal into an image actively hides it rather than revealing anything extra hiding in the pixels. If anything, the biggest lesson of this whole project was methodological, get your baselines right and audit your pipeline before trusting a fancy architecture's failure or success at face value.

How does this stack up against published work

Before calling 54.5% a finished number, I wanted to see how it compares to what's already out there. A few reference points from the literature on Dota draft prediction:

Conley and Perry (2013), one of the earliest papers in this space, got 69.8% test accuracy with plain Logistic Regression on picks alone, using about 18,000 mixed skill public matches.

Semenov et al. (2017), the most cited systematic benchmark in this exact area, tested Naive Bayes, Logistic Regression, Gradient Boosted Trees, and Factorization Machines on a large public dataset spanning multiple skill tiers, and explicitly found that accuracy depends heavily on the skill level of the players involved.

One genetic algorithm and logistic regression ensemble reported 74.1% accuracy, though it's worth noting the literature itself flags this result as shaky, since it was trained and tested on only 220 matches with no AUC reported alongside it.

A separate line of that same research adds hand engineered hero interaction features, matchup, synergy, and countering, on top of raw picks, and that pushes plain picks only Logistic Regression from around 69.4% up to about 72.9%.

So the honest comparison is that my 54.5% sits meaningfully below most of these published numbers, and that gap is worth digging into rather than glossing over.

What I'm missing compared to that published work

A few real differences jump out once you line the setups up side by side.

Dataset size. Most of that published work runs on 18,000 or more matches. I have 5,939. More data generally helps any model, especially a tree based or embedding based one, pick up on subtler hero interaction patterns across 127 plus heroes and the huge number of possible pairings between them.

Skill level and match type. My dataset is pro matches only, on a single patch, specifically to remove skill variance from the picture. Most of the published benchmarks pull from mixed skill public matchmaking, where skill gaps between players are much larger. That's actually an interesting distinction rather than just a shortcoming, if pro players are closer to equally skilled and coordinated, more of the outcome variance may genuinely come from in game execution rather than the draft itself, meaning a lower draft only ceiling in pro data could be a real finding, not a modeling failure. Worth holding onto that idea, but not using it as an excuse until I've actually tried closing the feature gap first.

The game itself has changed a lot since most of that published work. Conley and Perry is from 2013, Semenov et al. is from 2017. Dota has gone through enormous changes since then, a much bigger map, reworked gold and experience mechanics, entirely new items, years of hero reworks and balance passes. A lot of what made hard counters so decisive back then, certain heroes just flat out beating others regardless of skill, has been smoothed out over the years specifically because Valve keeps patching around those matchups. Comeback mechanics, catch up gold, and buyback changes also mean a bad draft matters less than it used to, since games have more built in ways to swing back from an early disadvantage. That doesn't necessarily mean my number should be lower than theirs, but it does mean directly comparing accuracy percentages across an eight to twelve year gap is a bit of an apples to oranges exercise. The hero pool, the map, and the core economy of the game are different enough that a synergy or counter matrix built from 2013 data may simply not describe how heroes interact anymore, and by extension a modern draft might genuinely carry less predictive weight than it did back then, independent of whether my feature engineering is good enough.

Missing hero interaction features. This is the big one. Every version of my pipeline so far, one hot GBM, learned embedding MLP, image based ViT and ResNet variants, only encodes which heroes are present. None of them explicitly encode how heroes interact with each other. The published work that clears 70% specifically adds synergy features (do these two heroes tend to win more often when picked together) and counter features (does this hero on one team tend to beat that hero on the other team historically). Those pairwise interaction terms are exactly the kind of signal a plain one hot vector or an averaged embedding pooling step can miss, since

summing or averaging embeddings together loses information about which specific pairs are interacting, it only keeps a blurred average of "who's present."

Still curious about

A few threads worth chasing next, some of these came directly out of the literature comparison above.

The literature points pretty directly at hero interaction features as a big part of the gap between the high 50s and the 69 to 72% range other researchers found. I haven't actually built this yet, but the plan would be to construct two matrices from training data only, to avoid leaking test information, a same team synergy matrix tracking how often each pair of heroes wins together when on the same side, and a counter matrix tracking how often a hero on one team's presence correlates with beating a specific hero on the other team. Each match would then get extra engineered features on top of the raw one hot vectors, a radiant synergy score, a dire synergy score, and a cross team counter score. If that meaningfully closes the gap toward 69 to 72%, it points at feature engineering as the main thing holding the baselines back. If it barely moves the needle, that's evidence the lower ceiling really is a property of pro only, single patch data, or of how much the game has changed since that older research.

Does adding ban data move the needle at all, since bans encode meta information picks alone don't capture.

With random hero ordering instead of role based ordering, does performance actually drop, meaning position secretly mattered, or stay flat, meaning it really is just about which heroes showed up.

Which heroes come out as most impactful in the embedding model, and does that line up with their real winrate on this patch.

Does grayscale or just edges and silhouettes perform any differently than full color icons, or is color a total non factor.

Would repeated cross validation tighten the confidence interval on that 54.5% number, and where would it actually land.

What information beyond the draft itself could be useful without crossing into actual gameplay data, something like match duration could be a proxy for certain heroes being stronger lategame than earlygame, without needing to peek at what actually happened during the game.

Still poking at these. If the draft really did decide the game before it even started, I'd expect a much bigger number than 54.5%. So maybe the truth is somewhere in between, the draft matters, just nowhere near as much as tilted post game chat would have you believe.

